# RELATÓRIO SOBRE O FRAMEWORK REACT

Principais características, vantagens, aplicações de mercado e exemplo prático

Relatório técnico-acadêmico

29 de outubro de 2026

# 1. INTRODUÇÃO

O desenvolvimento de aplicações web evoluiu significativamente com a popularização de interfaces dinâmicas, serviços on-line e dispositivos com diferentes tamanhos de tela. Nesse contexto, bibliotecas e frameworks de JavaScript ajudam a organizar a construção de interfaces, permitindo que equipes desenvolvam aplicações mais interativas e fáceis de manter. Uma das tecnologias mais utilizadas nesse cenário é o React.

O React é uma biblioteca JavaScript de código aberto voltada à criação de interfaces de usuário. Foi desenvolvido inicialmente no Facebook, atualmente Meta, e tornou-se uma opção relevante para aplicações que precisam atualizar partes da página conforme o usuário interage com o sistema. Embora seja frequentemente chamado de framework no uso cotidiano, sua classificação técnica mais precisa é biblioteca de interface: ele oferece mecanismos para construir a camada visual, enquanto roteamento, gerenciamento de estado global e comunicação com serviços podem ser resolvidos por ferramentas complementares.

Este relatório apresenta as principais características do React, suas vantagens e limitações, aplicações no mercado e um exemplo de utilização em uma aplicação web. O objetivo é explicar como a tecnologia funciona e em quais situações sua adoção pode ser adequada, considerando aspectos de desenvolvimento, manutenção, experiência do usuário e integração com back-end.

# 2. O QUE É O REACT

React é uma biblioteca para construir interfaces por meio de componentes reutilizáveis. Um componente representa uma parte da interface — por exemplo, um cabeçalho, um formulário de cadastro, um cartão de produto ou uma lista de clientes. Ao combinar componentes menores, é possível montar telas complexas com uma estrutura organizada.

A interface costuma ser descrita declarativamente: em vez de programar manualmente cada alteração do HTML, o desenvolvedor define como a tela deve se apresentar para determinado estado dos dados. Quando esse estado muda, o React calcula as alterações necessárias e atualiza a interface. Esse modelo ajuda a manter a visualização coerente com os dados da aplicação.

O React pode ser usado em páginas já existentes, em aplicações de página única (SPA) e em projetos que utilizam renderização no servidor ou geração estática por meio de soluções complementares. Também pode ser integrado a APIs desenvolvidas em tecnologias como Java, Spring Boot, Node.js, Python ou .NET. O React cuida principalmente da interface; regras de negócio, autenticação sensível e persistência normalmente ficam no servidor.

# 3. PRINCIPAIS CARACTERÍSTICAS

3.1 Componentes reutilizáveis. A interface é dividida em unidades menores, com responsabilidades definidas. Um componente de botão, por exemplo, pode ser reutilizado em várias telas, reduzindo duplicação e facilitando ajustes.

3.2 JSX. JSX é uma extensão de sintaxe que permite escrever estruturas semelhantes a HTML dentro de código JavaScript. O JSX é transformado durante o processo de compilação e facilita visualizar a relação entre a estrutura da interface e a lógica do componente.

3.3 Props e estado. Props são propriedades recebidas por um componente, normalmente de seu componente pai. O estado representa informações que podem mudar durante a execução, como o

texto digitado em um campo, os itens de um carrinho ou o resultado de uma consulta. Alterações no estado podem provocar uma nova renderização.

3.4 Hooks. Hooks são funções que permitem utilizar recursos do React em componentes funcionais. Entre os mais conhecidos estão useState, para estado local, e useEffect, para sincronizar o componente com sistemas externos, como uma API. Hooks personalizados também permitem compartilhar lógica entre componentes.

3.5 Atualização eficiente da interface. O React compara a representação anterior e a nova da interface e aplica as mudanças necessárias ao DOM. Esse processo é frequentemente associado ao conceito de Virtual DOM, mas não significa que toda atualização seja automaticamente rápida: o desempenho depende da arquitetura, do volume de dados e das decisões de implementação.

3.6 Fluxo de dados previsível. Em geral, os dados são passados dos componentes pais para os filhos por meio de props. Para compartilhar dados entre partes distantes da aplicação, podem ser usados Context, bibliotecas de estado ou outras estratégias. Um fluxo bem definido facilita compreender por que uma tela apresenta determinado resultado.

3.7 Ecossistema. O React conta com ferramentas complementares para roteamento, testes, formulários, acessibilidade, estilização e integração com APIs. Essa flexibilidade é uma característica importante, mas exige que a equipe escolha e mantenha as dependências adequadas.

3.8 Suporte a diferentes estratégias de renderização. Além da renderização no navegador, ferramentas do ecossistema permitem renderização no servidor e geração de páginas estáticas, úteis em contextos que valorizam carregamento inicial, compartilhamento de conteúdo e otimização para mecanismos de busca.

## 4. VANTAGENS DO REACT

Reutilização e padronização: componentes podem ser utilizados em diferentes páginas e funcionalidades. Isso reduz repetição de código e favorece a consistência visual, principalmente em sistemas com muitas telas.

Manutenção e organização: a divisão por componentes permite isolar responsabilidades. Quando um formulário precisa ser alterado, por exemplo, a equipe pode trabalhar no componente correspondente sem reescrever toda a página, desde que as dependências estejam bem definidas.

Interface interativa: o modelo declarativo e o gerenciamento de estado tornam mais simples implementar filtros, menus, validações, notificações, carrinhos e atualizações de dados sem recarregar a página inteira.

Ecossistema e comunidade: a adoção ampla oferece documentação, bibliotecas, exemplos e profissionais com experiência na tecnologia. Isso pode reduzir o tempo necessário para encontrar soluções, embora seja importante avaliar a qualidade e a manutenção de cada dependência.

Flexibilidade de integração: React pode consumir APIs REST ou GraphQL e trabalhar com back-ends existentes. Uma empresa pode manter um servidor em Java e desenvolver a camada de apresentação com React, comunicando-se por HTTP e formatos como JSON.

Escalabilidade organizacional: com convenções claras, componentes reutilizáveis, testes e padrões de projeto, diferentes pessoas podem trabalhar em partes distintas da interface. O ganho

depende da qualidade da arquitetura e dos processos da equipe.

Experiência do usuário: interfaces que respondem rapidamente às ações e exibem estados de carregamento, erro e sucesso podem tornar a utilização mais clara. React fornece ferramentas para implementar esse comportamento, mas a experiência final depende também de design, acessibilidade e desempenho.

## 5. LIMITAÇÕES E CUIDADOS

React não resolve sozinho todos os aspectos de uma aplicação. Roteamento, validação robusta, gerenciamento complexo de estado, autenticação e comunicação com serviços podem exigir ferramentas ou padrões adicionais. A liberdade de escolha é útil, mas pode gerar decisões inconsistentes se o projeto não tiver convenções.

A curva de aprendizagem inclui JavaScript moderno, JSX, componentes, estado, props, Hooks e ferramentas de desenvolvimento. Em equipes iniciantes, decisões inadequadas — como efeitos usados desnecessariamente, componentes muito grandes ou estado duplicado — podem aumentar a complexidade.

O tamanho do pacote enviado ao navegador, o número de renderizações e a quantidade de dependências precisam ser monitorados. Técnicas como divisão de código, carregamento sob demanda, memoização quando justificada e análise de desempenho ajudam, mas devem ser aplicadas com base em medições.

A acessibilidade não é automática. Elementos semânticos, navegação por teclado, rótulos de formulário, contraste e mensagens compreensíveis precisam ser considerados durante a implementação. Também é essencial evitar colocar segredos ou credenciais privadas no código enviado ao navegador.

## 6. APLICAÇÕES NO MERCADO

O React é utilizado em diferentes tipos de produtos digitais, especialmente quando há necessidade de interfaces interativas e reutilizáveis. Entre as aplicações comuns estão painéis administrativos, sistemas de gestão, plataformas de comércio eletrônico, portais de conteúdo, ferramentas de atendimento, redes sociais, sistemas educacionais e aplicações financeiras.

Em um painel administrativo, componentes podem apresentar indicadores, tabelas, filtros e formulários para cadastro. Em comércio eletrônico, o React pode compor catálogos, busca de produtos, carrinho e etapas de checkout. Em plataformas educacionais, pode controlar listas de cursos, progresso do estudante, questionários e feedback de respostas.

Em aplicações financeiras, a biblioteca pode exibir gráficos, saldos, transações e filtros por período. A segurança e a consistência dos dados, porém, devem ser garantidas no servidor; o front-end não deve ser considerado uma fronteira de segurança. Para conteúdo público que precisa ser indexado, a equipe pode avaliar soluções de renderização no servidor ou geração estática.

A escolha do React deve considerar o tamanho do projeto, o conhecimento da equipe, requisitos de desempenho, acessibilidade, SEO, manutenção e integração com sistemas existentes. Para páginas estáticas muito simples, HTML e CSS podem ser suficientes. Para interfaces ricas e com muitos estados, React pode trazer benefícios significativos.

# 7. EXEMPLO PRÁTICO: APLICAÇÃO WEB DE CADASTRO DE CLIENTES

Considere um sistema web de cadastro de clientes. A tela precisa apresentar um formulário com nome e e-mail, permitir adicionar registros e exibir a lista atualizada. A aplicação pode ser organizada em componentes como App, FormularioCliente e ListaClientes. Em um sistema real, os registros seriam enviados a uma API e persistidos em um banco de dados; para demonstrar o funcionamento do React, o exemplo a seguir mantém os dados apenas no estado do navegador.

O código utiliza useState para guardar os campos e a lista de clientes. Ao enviar o formulário, a função trata o evento, verifica se os campos estão preenchidos, adiciona o cliente ao estado e limpa os campos. Como a lista depende do estado, a interface é renderizada novamente com o novo registro.

Exemplo simplificado (arquivo App.jsx):

```
import { useState } from 'react';

export default function App() {
  const [nome, setNome] = useState('');
  const [email, setEmail] = useState('');
  const [clientes, setClientes] = useState([]);

  function cadastrarCliente(event) {
    event.preventDefault();

    if (!nome.trim() || !email.trim()) {
      alert('Preencha o nome e o e-mail.');
      return;
    }

    const novoCliente = {
      id: crypto.randomUUID(),
      nome: nome.trim(),
      email: email.trim()
    };

    setClientes((listaAtual) => [...listaAtual, novoCliente]);
    setNome('');
    setEmail('');
  }

  return (
    <main>
      <h1>Cadastro de clientes</h1>

      <form onSubmit={cadastrarCliente}>
        <label>
          Nome
          <input
            value={nome}
            onChange={(evento) => setNome(evento.target.value)}
            required
          />
        </label>

        <label>
          E-mail
          <input
            type="email"
            value={email}
            onChange={(evento) => setEmail(evento.target.value)}
            required
          />
        </label>

        <button type="submit">Cadastrar</button>
      </form>

      <h2>Clientes cadastrados</h2>
      {clientes.length === 0 ? (
```

```
          <p>Nenhum cliente cadastrado.</p>
        ) : (
          <ul>
            {clientes.map((cliente) => (
              <li key={cliente.id}>
                {cliente.nome} — {cliente.email}
              </li>
            ))}
          </ul>
        )}
      </main>
    );
}
```

No exemplo, value e onChange conectam os campos ao estado, criando campos controlados. O evento onSubmit chama a função de cadastro. A expressão clientes.map percorre a lista e apresenta um item para cada cliente; a propriedade key fornece uma identificação estável para cada elemento renderizado. crypto.randomUUID() gera um identificador para uso no exemplo, mas um sistema integrado a um banco de dados normalmente deve respeitar a estratégia de identificação definida pela API.

Para executar esse exemplo, pode-se criar um projeto React com uma ferramenta de inicialização adequada, colocar o código em App.jsx e iniciar o servidor de desenvolvimento conforme as instruções da ferramenta escolhida. O exemplo é demonstrativo: os dados são perdidos ao recarregar a página, não há persistência, autenticação ou comunicação com o servidor, e a validação apresentada é básica.

## 8. INTEGRAÇÃO COM UMA API E BOAS PRÁTICAS

Em uma aplicação completa, o componente de interface pode enviar os dados a um back-end por meio de uma requisição HTTP. Por exemplo, uma chamada POST para /api/clientes enviaria um objeto JSON com nome e e-mail. O servidor validaria os dados, aplicaria as regras de negócio, gravaria o registro no banco e retornaria uma resposta com o resultado. O React então poderia atualizar a lista ou apresentar uma mensagem de sucesso ou erro.

Boas práticas incluem separar componentes por responsabilidade, manter o estado o mais próximo possível de onde é utilizado, usar nomes claros, validar dados no cliente e no servidor, tratar falhas de rede e indicar estados de carregamento. Também convém escrever testes para comportamentos importantes e utilizar ferramentas de análise estática e formatação.

Em formulários reais, é importante apresentar rótulos acessíveis, mensagens de validação específicas e foco adequado após erros. Para listas grandes, pode ser necessário implementar paginação ou virtualização. Dados sensíveis não devem ser armazenados indiscriminadamente no navegador, e as permissões precisam ser verificadas no servidor, mesmo que a interface esconda determinadas opções.

## 9. CONCLUSÃO

React é uma biblioteca importante no desenvolvimento moderno de interfaces web. Seu modelo baseado em componentes, a composição de elementos, o uso de estado e o ecossistema de ferramentas permitem criar desde pequenos módulos interativos até aplicações de grande porte. Entre seus principais benefícios estão a reutilização de código, a organização da interface, a integração com APIs e o suporte de uma comunidade ampla.

Por outro lado, React não substitui a necessidade de uma arquitetura bem planejada. A equipe precisa selecionar ferramentas complementares, controlar dependências, cuidar do desempenho,

testar funcionalidades e desenvolver com acessibilidade e segurança. A escolha deve ser orientada pelos requisitos do projeto, e não apenas pela popularidade da tecnologia.

No exemplo de cadastro de clientes, foi possível observar como componentes, estado e eventos colaboram para atualizar uma interface conforme a interação do usuário. Em um ambiente de produção, o próximo passo seria integrar a tela a uma API e persistir os registros em um banco de dados, com validação e controle de acesso no servidor.

## 10. REFERÊNCIAS

REACT. Documentação oficial. Disponível em: https://react.dev/. Acesso em: 29 out. 2026.

META OPEN SOURCE. React — repositório oficial. Disponível em: https://github.com/facebook/react. Acesso em: 29 out. 2026.

MDN WEB DOCS. JavaScript. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript. Acesso em: 29 out. 2026.